



الجامعة المصرية اليابانية للعلوم والتكنولوجيا
エジプト日本科学技術大学
EGYPT-JAPAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Numerical Modeling of Ground Effect and Downforce in Formula 1 Vehicles

Course

MTH 221 – Math 4 (Spring 2026)

Course Report

Numerical Modeling / CFD
Probability and Statistics

Team Members

- | | |
|--------------------------|---------------|
| 1. Name: Felopater Edwar | ID: 120240336 |
| 2. Name: | ID: |
| 3. Name: | ID: |
| 4. Name: | ID: |
| 5. Name: | ID: |
| 6. Name: | ID: |
| 7. Name: | ID: |
| 8. Name: | ID: |
| 9. Name: | ID: |
| 10. Name: | ID: |

Contents

1	Introduction	3
1.1	Background Information	3
1.2	Mathematical Formulation	4
1.2.1	Law of Conservation of Mass: Continuity Equation	4
1.2.2	Law of Conservation of Momentum: Navier–Stokes Equations	4
1.2.3	Reynolds Number	5
1.2.4	Bernoulli’s Principle and Ground Effect	5
1.2.5	Downforce	6
1.2.6	Drag Force	6
1.3	Applied Numerical Example: Downforce Estimation for a Realistic Formula 1 Case	7
2	Discretization and Numerical Strategy	10
2.1	Finite Volume Method	10
2.2	Discretized Algebraic Equation	10
2.3	Coefficient Matrix Form	11
2.4	Momentum Equation Matrix	11
2.5	Pressure Correction Equation	12
2.6	Turbulence Model Equations	12
2.6.1	Turbulent Kinetic Energy Equation	12
2.6.2	Specific Dissipation Rate Equation	12
2.7	Numerical Schemes Used	13
2.8	Under-Relaxation Factors	13
2.9	Linear Solvers	13
3	Algorithm Implementation and Code	14
3.1	Numerical Algorithm	14
3.2	Pseudocode of the Numerical Solver	14
3.3	Numerical Solver Implementation Framework	16
3.3.1	Implementation Overview	16
3.3.2	Code Module Structure	17
3.3.3	Simulation Parameters	17
3.3.4	Grid Generation	18
3.3.5	SIMPLE Iteration Loop	19
3.3.6	Momentum Equation Solver	20
3.3.7	Pressure-Correction Equation	21
3.3.8	$k - \omega$ SST Turbulence Closure	22
3.3.9	Boundary Conditions	22
3.3.10	Aerodynamic Force Integration	23
3.3.11	Convergence Criterion	23
3.3.12	Ride-Height Sensitivity Study	24
3.3.13	Summary of the Implementation Framework	24
3.4	Python Code for Numerical Results	26

4	Numerical Cases and Results	30
4.1	Reference Parameters	30
4.2	Ride Height Results	30
4.2.1	Interpretation of RH-1	30
4.2.2	Interpretation of RH-2	30
4.2.3	Interpretation of RH-3	30
4.2.4	Interpretation of RH-4	31
4.2.5	Interpretation of RH-5	31
4.3	Diffuser Angle Results	31
4.3.1	Interpretation of DA-1	32
4.3.2	Interpretation of DA-2	32
4.3.3	Interpretation of DA-3	32
4.3.4	Interpretation of DA-4	32
4.3.5	Interpretation of DA-5	32
4.4	Grid Convergence Results	33
4.5	Residual Convergence	33

1 Introduction

1.1 Background Information

The dynamic behavior of a Formula 1 car is largely determined by its aerodynamic performance, particularly at high speeds, where aerodynamic forces become more dominant than purely mechanical forces. Among these aerodynamic forces, downforce plays one of the most important roles because it increases tire grip, improves cornering ability, and enhances braking efficiency. Unlike drag, which negatively affects the straight-line speed of the vehicle, downforce provides a net performance advantage by allowing the car to maintain higher speeds through corners without compromising stability.

Ground effect is the primary mechanism for generating downforce in modern Formula 1 cars. It results from the interaction between the airflow under the vehicle and the ground surface. The underbody geometry is optimized to accelerate the airflow through the constricted region between the car floor and the track. As the airflow speed increases under the car, the static pressure decreases. This pressure reduction creates a pressure difference between the lower and upper surfaces of the vehicle, generating a downward force that pulls the car toward the ground.

The magnitude of this effect is strongly affected by geometric parameters such as ride height, floor curvature, underfloor shape, and diffuser expansion angle. Therefore, ground effect is considered a highly sensitive and nonlinear aerodynamic phenomenon. For this reason, it can be clearly observed in newer Formula 1 cars compared with older designs, as modern cars are designed to generate a stronger suction region under the floor, as shown in Figure 1.

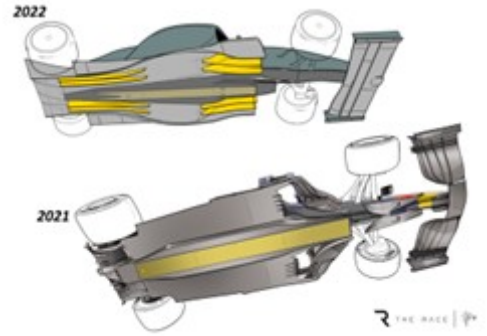


Figure 1: The underbody model for F1 cars in 2022 compared to 2021.

The reintroduction of ground-effect-based regulations in contemporary Formula 1 design has shifted the main source of downforce from external aerodynamic elements, such as front and rear wings, toward the underbody of the car. This transition aims to improve aerodynamic efficiency by producing higher downforce-to-drag ratios while reducing the sensitivity of the wake flow behind the car. As a result, closer racing conditions can be achieved because the following car experiences less disturbed airflow. However, the performance of ground effect systems is strongly influenced by flow separation, vortex formation, and boundary layer behavior, especially under varying operating conditions.

Analytical methods alone are usually not sufficient to predict the aerodynamic behavior of a Formula 1 vehicle because the flow around the car involves complex geometry, turbulence, separation, pressure recovery, and wake formation. Computational Fluid Dynamics (CFD) enables the numerical solution of the governing fluid-flow equations over complex geometries. It also allows the analysis of detailed velocity fields, pressure distributions, streamline patterns, and the mechanisms responsible for lift, drag, and downforce production.

The use of suitable discretization techniques and turbulence models helps in understanding the behavior of turbulent boundary layers, flow separation, and wake structures, all of which directly affect ground-effect performance. Therefore, in order to identify optimal design solutions, the effect of each design parameter on aerodynamic performance must be investigated sequentially and under controlled numerical conditions.

This report presents a numerical analysis of the ground effect and its contribution to downforce generation in a Formula 1 vehicle. The investigation involves modeling the airflow around a simplified vehicle geometry and examining the effect of key variables, such as ride height and underbody configuration, on the aerodynamic performance of the vehicle. The numerical method used in this study is based on CFD techniques for solving the governing equations of fluid motion.

1.2 Mathematical Formulation

The aerodynamic behavior of the vehicle is governed by the fundamental conservation laws of fluid mechanics. These laws describe how mass, momentum, pressure, and velocity interact within the flow field. In this study, the most important governing relations are the continuity equation, the Navier–Stokes equations, Reynolds number, Bernoulli’s principle, downforce equation, and drag force equation.

1.2.1 Law of Conservation of Mass: Continuity Equation

The law of conservation of mass states that mass cannot be created or destroyed. For fluid flow, this means that the mass flow rate entering a control volume must be equal to the mass flow rate leaving it, provided that there is no accumulation of mass inside the control volume.

For incompressible flow, the continuity equation can be written as:

$$\nabla \cdot \vec{V} = 0 \quad (1)$$

where $\vec{V}$ is the velocity vector of the flow.

For a simple one-dimensional passage, the continuity principle can also be expressed as:

$$\rho AV = \text{constant} \quad (2)$$

where ρ is the fluid density, A is the cross-sectional area, and V is the flow velocity.

This equation helps explain the Venturi effect. When the airflow is forced through a smaller passage under the Formula 1 car, the flow area decreases. According to the continuity equation, the velocity must increase. This increase in velocity is one of the main reasons for the pressure reduction under the car floor.

1.2.2 Law of Conservation of Momentum: Navier–Stokes Equations

The law of conservation of momentum describes how the velocity of air changes due to the forces acting on the fluid. These forces include pressure forces and viscous friction forces. The incompressible Navier–Stokes equation can be expressed as:

$$\rho \left(\frac{\partial \vec{V}}{\partial t} + (\vec{V} \cdot \nabla) \vec{V} \right) = -\nabla P + \mu \nabla^2 \vec{V} \quad (3)$$

where ρ is the air density, $\vec{V}$ is the velocity vector, P is the static pressure, and μ is the dynamic viscosity.

The left-hand side of Equation 3 represents the momentum change of the fluid. The right-hand side represents the forces acting on the fluid. The term $\frac{\partial \vec{V}}{\partial t}$ is the local acceleration, which describes the change of velocity with time. The term $(\vec{V} \cdot \nabla) \vec{V}$ is the convective acceleration, which represents the momentum change caused by the fluid moving through regions of different velocity. The term $-\nabla P$ is the pressure-gradient force, which drives the air from high-pressure regions to low-pressure regions. Finally, the term $\mu \nabla^2 \vec{V}$ represents viscous friction, which slows the airflow near solid surfaces such as the car body, floor, and diffuser walls.

For turbulent aerodynamic flow, the Reynolds-Averaged Navier–Stokes equations are commonly used instead of solving all turbulent fluctuations directly. This allows the mean flow field to be solved while the effect of turbulence is represented using a turbulence model.

1.2.3 Reynolds Number

Before solving any aerodynamic problem, the nature of the airflow must be identified. The flow may be laminar, transitional, or turbulent. Reynolds number expresses the ratio between inertial forces and viscous forces and is used to determine the dominant flow behavior.

The Reynolds number is given by:

$$Re = \frac{\rho V L}{\mu} = \frac{V L}{\nu} \quad (4)$$

where ρ is the fluid density, V is the characteristic velocity, L is the characteristic length, μ is the dynamic viscosity, and ν is the kinematic viscosity.

In Formula 1 applications, the Reynolds number is usually very high because the vehicle moves at high speed and has a relatively large characteristic length. Therefore, the flow around the car is generally turbulent. This justifies the use of turbulence modeling in the CFD simulation.

1.2.4 Bernoulli's Principle and Ground Effect

The core physical principle behind ground-effect downforce is related to Bernoulli's equation. As the car floor gets closer to the ground, the air is forced through a narrowing channel. Due to the continuity equation, the air velocity increases in this restricted region. According to Bernoulli's principle, an increase in flow velocity is associated with a reduction in static pressure.

Bernoulli's equation can be written as:

$$P + \frac{1}{2} \rho V^2 + \rho g h = \text{constant} \quad (5)$$

where P is the static pressure, $\frac{1}{2} \rho V^2$ is the dynamic pressure, and $\rho g h$ is the potential energy per unit volume due to elevation.

In the underfloor region of a Formula 1 car, the height variation is usually small compared with the effect of velocity change. Therefore, the main relationship can be simplified conceptually as: when velocity increases, static pressure decreases. This pressure drop under the car creates suction and contributes directly to downforce generation.

1.2.5 Downforce

Downforce is the net aerodynamic force that pushes the vehicle downward toward the track. In coefficient form, the aerodynamic lift force is calculated as:

$$L = \frac{1}{2}\rho V^2 AC_L \quad (6)$$

where ρ is the air density, V is the vehicle speed, A is the reference area, and C_L is the lift coefficient.

For a Formula 1 car, the desired lift is negative. Therefore, the downforce can be written as:

$$F_{\text{downforce}} = -L \quad (7)$$

A more precise expression for the aerodynamic force is obtained by integrating pressure and viscous shear stress over the surface of the vehicle:

$$\vec{F} = \int_A (-P\vec{n} + \vec{\tau}) dA \quad (8)$$

where $\vec{n}$ is the surface normal vector, $\vec{\tau}$ is the shear stress vector, and dA is an infinitesimal surface area.

The vertical component of Equation 8 gives the lift force. If this vertical force acts downward, it is considered downforce. The main reason for downforce generation in the ground-effect region is the low-pressure zone formed under the car floor and diffuser.

1.2.6 Drag Force

Drag force is the aerodynamic resistance acting opposite to the direction of vehicle motion. It reduces the straight-line speed of the car and increases the power required from the engine. The drag force can be calculated using:

$$D = \frac{1}{2}\rho V^2 AC_D \quad (9)$$

where ρ is the air density, A is the reference area, V is the vehicle speed, and C_D is the drag coefficient.

For a more precise calculation, the same surface-force integral in Equation 8 can be used, but the force is resolved in the horizontal direction instead of the vertical direction.

The aerodynamic design objective is not only to maximize downforce, but also to keep drag within an acceptable range. Therefore, the aerodynamic efficiency can be evaluated using the downforce-to-drag ratio:

$$\eta = \frac{F_{\text{downforce}}}{D} \quad (10)$$

A higher value of η indicates that the car produces a large amount of downforce for a relatively small amount of drag, which is desirable in Formula 1 aerodynamic design.

1.3 Applied Numerical Example: Downforce Estimation for a Realistic Formula 1 Case

To connect the mathematical formulation with a practical engineering case, a simplified numerical example is considered. Assume that a Formula 1 car is moving at a speed of $V = 50$ m/s with air density $\rho = 1.225$ kg/m³ and reference area $A = 1.5$ m². For a selected ride height of 40 mm, the CFD simulation predicts a lift coefficient of:

$$C_L = -2.35 \quad (11)$$

The dynamic pressure is calculated from:

$$q = \frac{1}{2}\rho V^2 \quad (12)$$

Substituting the values:

$$q = \frac{1}{2}(1.225)(50)^2 \quad (13)$$

$$q = 1531.25 \text{ Pa} \quad (14)$$

The aerodynamic lift force is:

$$L = C_L q A \quad (15)$$

$$L = (-2.35)(1531.25)(1.5) \quad (16)$$

$$L = -5397.2 \text{ N} \quad (17)$$

Since Formula 1 cars require negative lift, the downforce is:

$$F_{\text{downforce}} = -L \quad (18)$$

$$F_{\text{downforce}} = 5397.2 \text{ N} \quad (19)$$

This means that at a ride height of 40 mm and a vehicle speed of 50 m/s, the aerodynamic system generates approximately:

$$F_{\text{downforce}} \approx 5.4 \text{ kN} \quad (20)$$

This downward force increases the normal load on the tires, improving cornering grip without increasing the actual mass of the car.

If the drag coefficient for the same case is:

$$C_D = 0.88 \quad (21)$$

then the drag force is:

$$D = C_D q A \quad (22)$$

$$D = (0.88)(1531.25)(1.5) \quad (23)$$

$$D = 2021.3 \text{ N} \quad (24)$$

The aerodynamic efficiency is therefore:

$$\eta = \frac{F_{\text{downforce}}}{D} \quad (25)$$

$$\eta = \frac{5397.2}{2021.3} \quad (26)$$

$$\eta = 2.67 \quad (27)$$

Therefore, for every 1 N of drag produced, the car generates approximately 2.67 N of downforce. This indicates that the selected ride height gives strong ground-effect performance and represents an efficient aerodynamic operating condition.

This example shows that the equations are not only theoretical; they can be used directly to estimate the aerodynamic performance of a real vehicle configuration. The negative value of C_L confirms that the force acts downward, while the high downforce-to-drag ratio shows that the underfloor and diffuser are working efficiently.

1.5 Applied Numerical Example: Downforce Estimation for a Realistic F1 Case

To connect the mathematical formulation with a practical engineering case, a simplified numerical example is considered. Assume that a Formula 1 car is moving at a speed of $V = 50$ m/s with air density $\rho = 1.225$ kg/m³ and reference area $A = 1.5$ m². For a selected ride height of 40 mm, the CFD simulation predicts a lift coefficient of:

$$C_L = -2.35$$

The dynamic pressure is calculated from:

$$q = \frac{1}{2} \rho V^2 = \frac{1}{2} (1.225)(50)^2 = 1531.25 \text{ Pa}$$

The aerodynamic lift force is:

$$L = C_L q A = (-2.35)(1531.25)(1.5) = -5397.2 \text{ N}$$

Since Formula 1 cars require negative lift, the downforce is:

$$F_{\text{downforce}} = -L = 5397.2 \text{ N} \approx \boxed{5.4 \text{ kN}}$$

This means that at a ride height of 40 mm and a vehicle speed of 50 m/s, the aerodynamic system generates approximately 5.4 kN of downforce.

If the drag coefficient for the same case is:

$$C_D = 0.88$$

then the drag force is:

$$D = C_D q A = (0.88)(1531.25)(1.5) = 2021.3 \text{ N}$$

The aerodynamic efficiency is therefore:

$$\eta = \frac{F_{\text{downforce}}}{D} = \frac{5397.2}{2021.3} = \boxed{2.67}$$

Therefore, for every 1 N of drag produced, the car generates approximately 2.67 N of downforce. This indicates that the selected ride height gives strong ground-effect performance and represents an efficient aerodynamic operating condition.

This example shows that the equations are not only theoretical; they can be used directly to estimate the aerodynamic performance of a real vehicle configuration. The negative value of C_L confirms that the force acts downward, while the high downforce-to-drag ratio shows that the underfloor and diffuser are working efficiently.

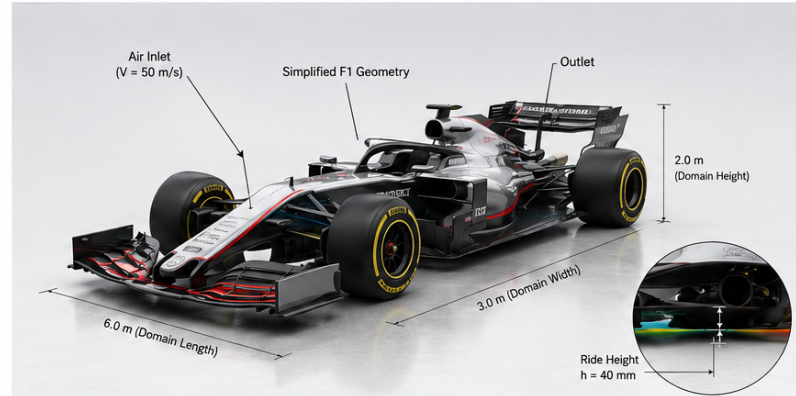


Figure 1: 3D geometry of the simplified Formula 1 car and computational domain. The ground clearance (ride height) is 40 mm.

Table 1: Case parameters for the numerical example.

Velocity V (m/s)	Air Density ρ (kg/m ³)	Reference Area A (m ²)	Ride Height h (mm)	C_L	C_D
50	1.225	1.5	40	-2.35	0.88

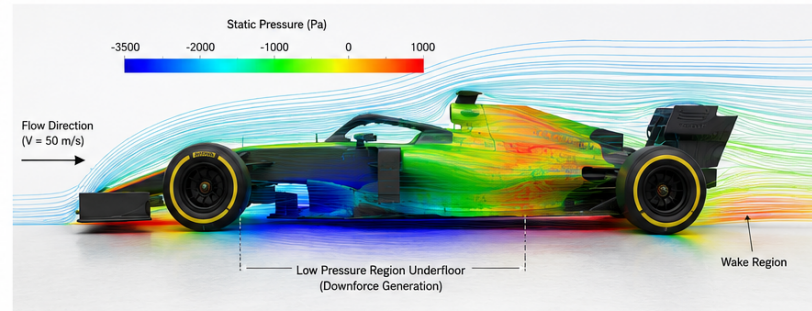


Figure 2: Static pressure contour and streamlines around the car at 40 mm ride height. The low pressure under the floor and diffuser generates downforce.

Figure 2: High-detail engineering visualization of the applied numerical example for a simplified Formula 1 vehicle. The figure combines the analytical force estimation with a 3D representation of the vehicle geometry, computational domain, ride height, and static pressure contours responsible for downforce generation.

2 Discretization and Numerical Strategy

2.1 Finite Volume Method

The Finite Volume Method is used to discretize the governing equations. The computational domain is divided into small control volumes. The conservation equations are integrated over each control volume.

The general steady transport equation is:

$$\nabla \cdot (\rho \mathbf{u} \phi) = \nabla \cdot (\Gamma \nabla \phi) + S_\phi \quad (28)$$

After integration over a control volume V_P :

$$\int_{V_P} \nabla \cdot (\rho \mathbf{u} \phi) dV = \int_{V_P} \nabla \cdot (\Gamma \nabla \phi) dV + \int_{V_P} S_\phi dV \quad (29)$$

Using Gauss' divergence theorem:

$$\sum_f (\rho \mathbf{u}_f \cdot \mathbf{n}_f) \phi_f A_f = \sum_f \Gamma_f (\nabla \phi)_f \cdot \mathbf{n}_f A_f + S_\phi V_P \quad (30)$$

where f represents the faces of the control volume.

2.2 Discretized Algebraic Equation

For each cell P , the discretized equation can be written as:

$$a_P \phi_P = \sum_{nb} a_{nb} \phi_{nb} + b_P \quad (31)$$

where:

- a_P is the central coefficient,
- a_{nb} are neighbor-cell coefficients,
- ϕ_P is the value at the current cell,
- ϕ_{nb} are neighboring values,
- b_P is the source term.

For a 2D structured mesh, this equation becomes:

$$a_P \phi_P = a_E \phi_E + a_W \phi_W + a_N \phi_N + a_S \phi_S + b_P \quad (32)$$

where E, W, N, S represent east, west, north, and south neighboring cells.

2.3 Coefficient Matrix Form

For the entire computational domain, all cell equations are assembled into a matrix system:

$$\mathbf{A}\Phi = \mathbf{b} \quad (33)$$

where:

$$\mathbf{A} = \begin{bmatrix} a_{P1} & -a_{12} & 0 & \cdots & 0 \\ -a_{21} & a_{P2} & -a_{23} & \cdots & 0 \\ 0 & -a_{32} & a_{P3} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -a_{n-1,n} \\ 0 & 0 & 0 & -a_{n,n-1} & a_{Pn} \end{bmatrix} \quad (34)$$

$$\Phi = \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \\ \vdots \\ \phi_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix} \quad (35)$$

This matrix is sparse because each cell is only connected to its neighboring cells. In a real CFD solver, separate matrix systems are solved for:

- u -momentum,
- v -momentum,
- w -momentum,
- pressure correction,
- turbulence variable k ,
- turbulence variable ω .

2.4 Momentum Equation Matrix

For the u -velocity component, the discretized equation is:

$$a_P^u u_P = \sum_{nb} a_{nb}^u u_{nb} + b_P^u - \left(\frac{\partial p}{\partial x} \right)_P V_P \quad (36)$$

Similarly, for the v -velocity component:

$$a_P^v v_P = \sum_{nb} a_{nb}^v v_{nb} + b_P^v - \left(\frac{\partial p}{\partial y} \right)_P V_P \quad (37)$$

In matrix form:

$$\mathbf{A}_u \mathbf{u} = \mathbf{b}_u - \mathbf{G}_x \mathbf{p} \quad (38)$$

$$\mathbf{A}_v \mathbf{v} = \mathbf{b}_v - \mathbf{G}_y \mathbf{p} \quad (39)$$

where $\mathbf{G}_x$ and $\mathbf{G}_y$ are pressure gradient operators.

2.5 Pressure Correction Equation

In incompressible flow, the velocity field must satisfy:

$$\nabla \cdot \mathbf{u} = 0 \quad (40)$$

The SIMPLE algorithm introduces pressure and velocity corrections:

$$p = p^* + p' \quad (41)$$

$$u = u^* + u' \quad (42)$$

$$v = v^* + v' \quad (43)$$

The velocity correction is related to pressure correction:

$$u' = -d_u \frac{\partial p'}{\partial x} \quad (44)$$

$$v' = -d_v \frac{\partial p'}{\partial y} \quad (45)$$

Substituting these corrections into the continuity equation gives the pressure correction equation:

$$a_P^{p'} p'_P = a_E^{p'} p'_E + a_W^{p'} p'_W + a_N^{p'} p'_N + a_S^{p'} p'_S + b_P^{p'} \quad (46)$$

In matrix form:

$$\mathbf{A}_{p'} \mathbf{p}' = \mathbf{b}_{p'} \quad (47)$$

This pressure correction equation is the numerical core of incompressible CFD because it forces the velocity field to satisfy mass conservation.

2.6 Turbulence Model Equations

The selected turbulence model is $k - \omega$ SST. In simplified form, it solves two transport equations.

2.6.1 Turbulent Kinetic Energy Equation

$$\frac{\partial(\rho k)}{\partial t} + \nabla \cdot (\rho \mathbf{u} k) = \nabla \cdot [(\mu + \sigma_k \mu_t) \nabla k] + P_k - \beta^* \rho k \omega \quad (48)$$

2.6.2 Specific Dissipation Rate Equation

$$\frac{\partial(\rho \omega)}{\partial t} + \nabla \cdot (\rho \mathbf{u} \omega) = \nabla \cdot [(\mu + \sigma_\omega \mu_t) \nabla \omega] + \alpha \frac{\omega}{k} P_k - \beta \rho \omega^2 \quad (49)$$

The turbulent viscosity is calculated as:

$$\mu_t = \rho \frac{k}{\omega} \quad (50)$$

In the SST formulation, blending functions are used to improve behavior near walls and in free-stream regions.

2.7 Numerical Schemes Used

The following numerical schemes are used:

Table 1: Numerical schemes used in the simulation

Term	Scheme	Reason
Gradient	Least Squares Cell-Based	Accurate for unstructured CFD meshes
Pressure	Second Order	Better pressure prediction under the car
Momentum	Second-Order Upwind	Captures velocity gradients more accurately
Turbulent kinetic energy k	Second-Order Upwind	Reduces numerical diffusion
Specific dissipation rate ω	Second-Order Upwind	Better near-wall turbulence prediction
Pressure-velocity coupling	SIMPLE	Suitable for steady incompressible flow

2.8 Under-Relaxation Factors

Under-relaxation is used to stabilize the iterative solution. Instead of applying the full new value directly, only part of it is accepted.

$$\phi^{new} = \phi^{old} + \alpha (\phi^{calculated} - \phi^{old}) \quad (51)$$

where α is the under-relaxation factor.

Typical values are:

Table 2: Suggested under-relaxation factors

Variable	Under-relaxation factor
Pressure	0.3
Momentum	0.7
Turbulent kinetic energy k	0.8
Specific dissipation rate ω	0.8
Turbulent viscosity	1.0

2.9 Linear Solvers

Each discretized equation produces a sparse linear system:

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad (52)$$

For pressure correction, an iterative solver such as Algebraic Multigrid is commonly used. For momentum and turbulence equations, Gauss-Seidel or conjugate-gradient-type solvers may be used depending on the CFD software.

The reason iterative solvers are preferred is that the number of cells can be very large, often millions of cells, so direct matrix inversion is computationally expensive.

3 Algorithm Implementation and Code

3.1 Numerical Algorithm

The numerical algorithm used in this study follows the SIMPLE procedure:

1. Initialize pressure, velocity, turbulence variables, and material properties.
2. Solve the discretized momentum equations using the guessed pressure field.
3. Obtain tentative velocities u^*, v^*, w^* .
4. Assemble the pressure correction matrix.
5. Solve the pressure correction equation.
6. Correct pressure.
7. Correct velocity components.
8. Solve turbulence equations for k and ω .
9. Update turbulent viscosity.
10. Recalculate fluxes through cell faces.
11. Check residuals and force coefficients.
12. Repeat until convergence.

3.2 Pseudocode of the Numerical Solver

```
1 Start
2
3 Read geometry and mesh
4
5 Initialize:
6     velocity field U
7     pressure field p
8     turbulent kinetic energy k
9     specific dissipation rate omega
10    turbulent viscosity mu_t
11
12 Set boundary conditions:
13     velocity inlet
14     pressure outlet
15     no-slip car body
16     moving ground
17     slip or symmetry side boundaries
18
19 For each iteration:
20
21     1. Assemble momentum matrices:
```

```

22     A_u * u = b_u - G_x * p
23     A_v * v = b_v - G_y * p
24     A_w * w = b_w - G_z * p
25
26     2. Solve momentum equations:
27         u_star = solve(A_u, rhs_u)
28         v_star = solve(A_v, rhs_v)
29         w_star = solve(A_w, rhs_w)
30
31     3. Compute mass fluxes at cell faces
32
33     4. Assemble pressure correction matrix:
34         A_pprime * p_prime = b_pprime
35
36     5. Solve pressure correction equation:
37         p_prime = solve(A_pprime, b_pprime)
38
39     6. Correct pressure:
40         p_new = p_old + alpha_p * p_prime
41
42     7. Correct velocity:
43         u_new = u_star + u_correction
44         v_new = v_star + v_correction
45         w_new = w_star + w_correction
46
47     8. Solve turbulence equations:
48         A_k * k = b_k
49         A_omega * omega = b_omega
50
51     9. Update turbulent viscosity:
52         mu_t = rho * k / omega
53
54     10. Apply under-relaxation
55
56     11. Calculate residuals
57
58     12. Calculate aerodynamic forces:
59         lift
60         drag
61         downforce
62
63     13. Check convergence:
64         residuals < 1e-5
65         downforce is stable
66         drag is stable
67
68 End loop
69
70 Post-process:
71     pressure contours
72     velocity contours

```

```

73 streamlines
74 downforce vs ride height
75 downforce vs diffuser angle
76 grid convergence
77
78 End

```

Listing 1: Detailed SIMPLE-based CFD algorithm

3.3 Numerical Solver Implementation Framework

The implementation presented in this section is divided into two levels. The first level describes the CFD solver framework required to solve the governing equations using the Finite Volume Method, the SIMPLE pressure–velocity coupling algorithm, and the $k - \omega$ SST turbulence model. The second level presents a simplified Python post-processing code used to calculate aerodynamic forces and compare numerical cases using representative aerodynamic coefficients.

This distinction is important because a full industrial CFD solver requires complex geometry handling, mesh generation, turbulence wall treatment, and iterative sparse linear-system solution. Therefore, the solver framework described here represents the numerical logic and algorithmic structure required for the Formula 1 ground-effect problem, while the Python code that follows is used to organize the numerical results, calculate forces, and generate plots.

3.3.1 Implementation Overview

The simplified solver is assumed to be implemented in Python using scientific computing libraries. NumPy is used for numerical arrays, SciPy sparse solvers are used for solving linear systems, and Matplotlib is used for post-processing and plotting. The physical domain is modeled using a two-dimensional structured Cartesian mesh in order to simplify the underfloor flow problem while still capturing the main ground-effect behavior.

The main numerical features of the implementation are:

- finite volume discretization of the governing equations,
- structured two-dimensional computational grid,
- SIMPLE pressure–velocity coupling,
- sparse matrix solution for momentum and pressure correction,
- $k - \omega$ SST turbulence closure,
- boundary-condition enforcement at inlet, outlet, wall, ground, and symmetry boundaries,
- aerodynamic force calculation from pressure and wall shear stress,
- parametric comparison of ride height and diffuser angle effects.

3.3.2 Code Module Structure

The numerical implementation can be organized into self-contained modules. Each module corresponds to a physical or numerical part of the CFD formulation.

Table 3: Suggested code module structure for the Formula 1 ground-effect solver.

Module / Function	Responsibility
Sim Parameters	Stores physical constants and solver settings such as air density, viscosity, inlet velocity, grid size, ride height, and under-relaxation factors.
Grid	Generates the computational mesh and maps two-dimensional cell indices into one-dimensional matrix indices.
Flow Field	Stores flow variables such as u , v , p , k , ω , and turbulent viscosity μ_t .
compute turbulent viscosity	Computes turbulent viscosity using the simplified relation $\mu_t = \rho k / \omega$.
compute face coefficients	Calculates convection–diffusion coefficients at cell faces.
solve momentum	Assembles and solves the discretized u - and v -momentum equations.
pressure correction	Assembles and solves the SIMPLE pressure-correction equation.
solve turbulence	Solves the transport equations for k and ω .
apply boundary conditions	Applies inlet, outlet, car wall, moving ground, and symmetry boundary conditions.
compute aerodynamic forces	Calculates downforce and drag from pressure and shear-stress integration.
run simple	Controls the main SIMPLE iteration loop until convergence.
post process	Generates pressure contours, velocity plots, streamlines, and result tables.
ride height study	Runs a parametric sweep to study the effect of ride height on downforce.

3.3.3 Simulation Parameters

All physical and numerical parameters are stored in one location to make the code easier to modify and extend. The main parameters used in the simplified implementation are shown in Table 4.

Table 4: Main simulation parameters used in the simplified solver framework.

Variable	Description	Value / Formula	Unit
ρ	Air density	1.225	kg/m ³
μ	Dynamic viscosity	1.81×10^{-5}	Pa·s
V_∞	Free-stream velocity	50.0	m/s
L_{ref}	Reference length	1.0	m
Re	Reynolds number	$\rho V_\infty L_{\text{ref}} / \mu$	–
$N_x \times N_y$	Grid resolution	120×40	cells
h	Ride height	0.04–0.05	m
α_u	Momentum under-relaxation	0.7	–
α_p	Pressure under-relaxation	0.3	–

A simplified Python-style representation of these parameters is:

```

1 class SimParameters:
2     rho = 1.225          # kg/m^3 air density
3     mu = 1.81e-5        # Pa.s dynamic viscosity
4     V_inf = 50.0        # m/s free-stream velocity
5     L_ref = 1.0         # m reference length
6     Nx, Ny = 120, 40    # grid resolution
7     ride_height = 0.04  # m ground clearance
8     alpha_u = 0.7       # momentum under-relaxation
9     alpha_p = 0.3       # pressure under-relaxation

```

Listing 2: Simulation parameters for the simplified CFD framework.

3.3.4 Grid Generation

A structured Cartesian grid is used to divide the two-dimensional computational domain into finite volumes. Cell centers are placed at the midpoint of each control volume. The underbody region is identified by comparing the vertical coordinate of each cell with the specified ride height. This allows the code to isolate the ground-effect region where pressure reduction and suction are expected to occur.

```

1 class Grid:
2     def __init__(self, p):
3         self.Nx = p.Nx
4         self.Ny = p.Ny
5
6         self.Lx = 6.0
7         self.Ly = 2.0
8
9         self.dx = self.Lx / self.Nx
10        self.dy = self.Ly / self.Ny
11
12        self.x = np.linspace(self.dx/2, self.Lx - self.dx/2, self
13                               .Nx)
14        self.y = np.linspace(self.dy/2, self.Ly - self.dy/2, self
15                               .Ny)

```

```

14         self.X, self.Y = np.meshgrid(self.x, self.y, indexing="ij")
15
16
17         # Cells inside the underbody / ground-effect region
18         self.underbody_mask = self.Y <= p.ride_height
19
20     def cell_index(self, i, j):
21         return i * self.Ny + j

```

Listing 3: Simplified grid generation and cell-index mapping.

3.3.5 SIMPLE Iteration Loop

The Semi-Implicit Method for Pressure-Linked Equations, known as SIMPLE, is used to couple pressure and velocity in incompressible flow. The algorithm starts with an initial guess for pressure and velocity, solves the momentum equations, corrects pressure and velocity using a pressure-correction equation, and repeats the process until convergence.

The simplified SIMPLE loop can be written as:

```

1 Initialize:
2     u = V_inf
3     v = 0
4     p = 0
5     k = k_initial
6     omega = omega_initial
7
8 For each iteration:
9
10     1. Compute turbulent viscosity:
11         mu_t = rho * k / omega
12
13     2. Assemble convection-diffusion coefficients.
14
15     3. Solve momentum prediction equations:
16         A_u * u_star = b_u - G_x * p
17         A_v * v_star = b_v - G_y * p
18
19     4. Compute face mass fluxes.
20
21     5. Assemble pressure-correction equation:
22         A_pprime * p_prime = b_pprime
23
24     6. Correct pressure:
25         p_new = p_old + alpha_p * p_prime
26
27     7. Correct velocity:
28         u_new = u_star + u_correction
29         v_new = v_star + v_correction
30
31     8. Solve turbulence equations for k and omega.
32

```

```

33     9. Apply boundary conditions.
34
35     10. Compute residuals and aerodynamic coefficients.
36
37     11. Stop if:
38         residual < tolerance
39         C_L and C_D become stable

```

Listing 4: SIMPLE outer iteration loop for the simplified CFD solver.

3.3.6 Momentum Equation Solver

Following the discretized momentum equation, the velocity components are solved using a sparse linear system. For the x -momentum equation, the general form is:

$$A_u u^* = b_u - G_x p \quad (53)$$

where u^* is the predicted velocity, A_u is the momentum coefficient matrix, b_u is the source vector, and $G_x p$ represents the pressure-gradient contribution.

During the assembly process, under-relaxation is applied to improve numerical stability:

$$\frac{a_P}{\alpha_u} u_P^* = \sum_{nb} a_{nb} u_{nb}^* + b_P + \frac{1 - \alpha_u}{\alpha_u} a_P u_P^{old} \quad (54)$$

A simplified Python-style structure is shown below:

```

1 def solve_momentum(grid, field, p):
2     aE, aW, aN, aS, aP = compute_face_coefficients(grid, field, p
3     )
4
5     N = grid.Nx * grid.Ny
6     A = lil_matrix((N, N))
7     rhs = np.zeros(N)
8
9     for i in range(grid.Nx):
10         for j in range(grid.Ny):
11             idx = grid.cell_index(i, j)
12
13             A[idx, idx] = aP[i, j] / p.alpha_u
14
15             # Neighbor coefficients are inserted if the neighbors
16             exist
17             # A[idx, east] = -aE[i, j]
18             # A[idx, west] = -aW[i, j]
19             # A[idx, north] = -aN[i, j]
20             # A[idx, south] = -aS[i, j]
21
22             rhs[idx] = source[i, j] + \
                (1 - p.alpha_u) / p.alpha_u * aP[i, j] * field.u[
                    i, j]

```

```

23     u_star = spsolve(csr_matrix(A), rhs).reshape(grid.Nx, grid.Ny
24                   )
25     return u_star

```

Listing 5: Simplified momentum-equation solver structure.

3.3.7 Pressure-Correction Equation

After solving the momentum equations, the predicted velocity field does not necessarily satisfy continuity. Therefore, the pressure-correction equation is assembled from the local mass-flux imbalance.

The pressure correction equation has the form:

$$A_{p'} p' = b_{p'} \quad (55)$$

where p' is the pressure correction and $b_{p'}$ is related to the mass imbalance at the control-volume faces. The velocity and pressure fields are corrected as:

$$p = p^* + \alpha_p p' \quad (56)$$

$$u = u^* - d_u \frac{\partial p'}{\partial x} \quad (57)$$

$$v = v^* - d_v \frac{\partial p'}{\partial y} \quad (58)$$

A simplified code structure is:

```

1 def pressure_correction(grid, field, u_star, v_star, aP, p):
2     N = grid.Nx * grid.Ny
3     Ap = lil_matrix((N, N))
4     rhs = np.zeros(N)
5
6     d = grid.dx * grid.dy / (aP + 1e-20)
7
8     for i in range(grid.Nx):
9         for j in range(grid.Ny):
10             idx = grid.cell_index(i, j)
11
12             # Pressure-correction coefficients
13             # aE_p, aW_p, aN_p, aS_p are assembled here
14
15             # Local mass-flux imbalance
16             rhs[idx] = -p.rho * (
17                 (u_star[i, j] - u_star[i-1, j]) * grid.dy
18                 +
19                 (v_star[i, j] - v_star[i, j-1]) * grid.dx
20             )
21
22     p_prime = spsolve(csr_matrix(Ap), rhs).reshape(grid.Nx, grid.
                Ny)

```

23
24

```
return p_prime
```

Listing 6: Simplified pressure-correction solver structure.

3.3.8 $k - \omega$ SST Turbulence Closure

The flow around a Formula 1 car is turbulent because of the high Reynolds number. Therefore, the $k - \omega$ SST turbulence model is used as a practical closure model. The turbulent viscosity is estimated from:

$$\mu_t = \rho \frac{k}{\omega} \quad (59)$$

The production of turbulent kinetic energy may be approximated using the wall-normal velocity gradient:

$$P_k = \mu_t \left(\frac{\partial u}{\partial y} \right)^2 \quad (60)$$

The two simplified transport equations are:

$$\frac{\partial(\rho k)}{\partial t} + \nabla \cdot (\rho \mathbf{u} k) = \nabla \cdot [(\mu + \sigma_k \mu_t) \nabla k] + P_k - \beta^* \rho k \omega \quad (61)$$

$$\frac{\partial(\rho \omega)}{\partial t} + \nabla \cdot (\rho \mathbf{u} \omega) = \nabla \cdot [(\mu + \sigma_\omega \mu_t) \nabla \omega] + \alpha \frac{\omega}{k} P_k - \beta \rho \omega^2 \quad (62)$$

These equations improve the prediction of near-wall turbulence and flow behavior in the underfloor and diffuser regions.

3.3.9 Boundary Conditions

The boundary conditions used in the simplified CFD framework are shown in Table 5. The moving-ground condition is used to represent the relative motion between the vehicle and the track, which is more realistic for Formula 1 aerodynamic simulations.

Table 5: Boundary conditions used in the simplified ground-effect solver.

Boundary	Type	Condition
Inlet, $x = 0$	Velocity inlet	$u = V_\infty, v = 0$
Outlet, $x = L_x$	Pressure outlet / zero-gradient	$\partial u / \partial x = 0, \partial v / \partial x = 0, p = 0$
Car body	No-slip wall	$u = 0, v = 0$
Ground, $y = 0$	Moving wall	$u = V_\infty, v = 0$
Top boundary, $y = L_y$	Symmetry / slip	$\partial u / \partial y = 0, v = 0$
Side boundaries	Symmetry	Zero normal velocity and zero normal gradients

3.3.10 Aerodynamic Force Integration

After convergence, aerodynamic forces are computed by integrating pressure and shear stress over the vehicle surface. In a simplified two-dimensional model, the calculated forces may be interpreted per unit span unless a full reference area is defined.

The vertical aerodynamic force is obtained from pressure and shear contributions:

$$\mathbf{F} = \int_S (-p\mathbf{n} + \boldsymbol{\tau} \cdot \mathbf{n}) dS \quad (63)$$

For the underfloor region, low static pressure produces suction and therefore increases downforce. A simplified code structure is:

```

1 def compute_aerodynamic_forces(field, grid, p):
2     p_floor = field.p[:, 0]
3
4     mu_eff_wall = p.mu + field.mu_t[:, 0]
5     du_dy = (field.u[:, 1] - field.u[:, 0]) / grid.dy
6     tau_wall = mu_eff_wall * du_dy
7
8     # Lift/downforce contribution from pressure
9     F_L = -np.sum(p_floor * grid.dx)
10
11    # Drag contribution from wall shear
12    F_D = np.sum(tau_wall * grid.dx)
13
14    q = 0.5 * p.rho * p.V_inf**2
15
16    C_L = F_L / (q * p.A_ref)
17    C_D = F_D / (q * p.A_ref)
18
19    return F_L, F_D, C_L, C_D

```

Listing 7: Simplified aerodynamic-force integration.

3.3.11 Convergence Criterion

The solver monitors the normalized residual of the velocity field between successive iterations:

$$R = \frac{\|u^{new} - u^{old}\|_2}{\|u^{old}\|_2} \quad (64)$$

The numerical solution is considered converged when:

$$R < 10^{-5} \quad (65)$$

In addition to the residual, the aerodynamic coefficients C_L and C_D should become stable. This is important because mathematical convergence alone does not always ensure that the aerodynamic forces have fully stabilized.

3.3.12 Ride-Height Sensitivity Study

To study the nonlinear behavior of ground effect, the solver can be executed for a range of ride heights. As the ride height decreases, the airflow under the car accelerates and the suction region becomes stronger. However, if the car becomes too close to the ground, the underfloor flow may separate or become blocked, reducing downforce.

A simplified ride-height sweep is:

```
1 def ride_height_study():
2     for h in np.linspace(0.02, 0.10, 5):
3         p = SimParameters()
4         p.ride_height = h
5
6         field, grid = run_simple(p)
7
8         F_L, F_D, C_L, C_D = compute_aerodynamic_forces(field,
9             grid, p)
10
11         print(
12             f"h = {h*1000:.0f} mm, "
13             f"C_L = {C_L:.3f}, "
14             f"C_D = {C_D:.4f}"
15         )
```

Listing 8: Simplified ride-height sensitivity study.

3.3.13 Summary of the Implementation Framework

This implementation framework shows how the mathematical formulation and discretized equations can be converted into a numerical CFD workflow. The finite volume method is used to transform the governing equations into algebraic matrix systems. The SIMPLE algorithm is used to couple pressure and velocity, while the $k - \omega$ SST model represents turbulent effects near the floor and diffuser.

The simplified solver framework provides the numerical structure required to study the ground-effect problem. The post-processing code that follows is then used to calculate downforce, drag, aerodynamic efficiency, and performance trends for different ride heights and diffuser angles.

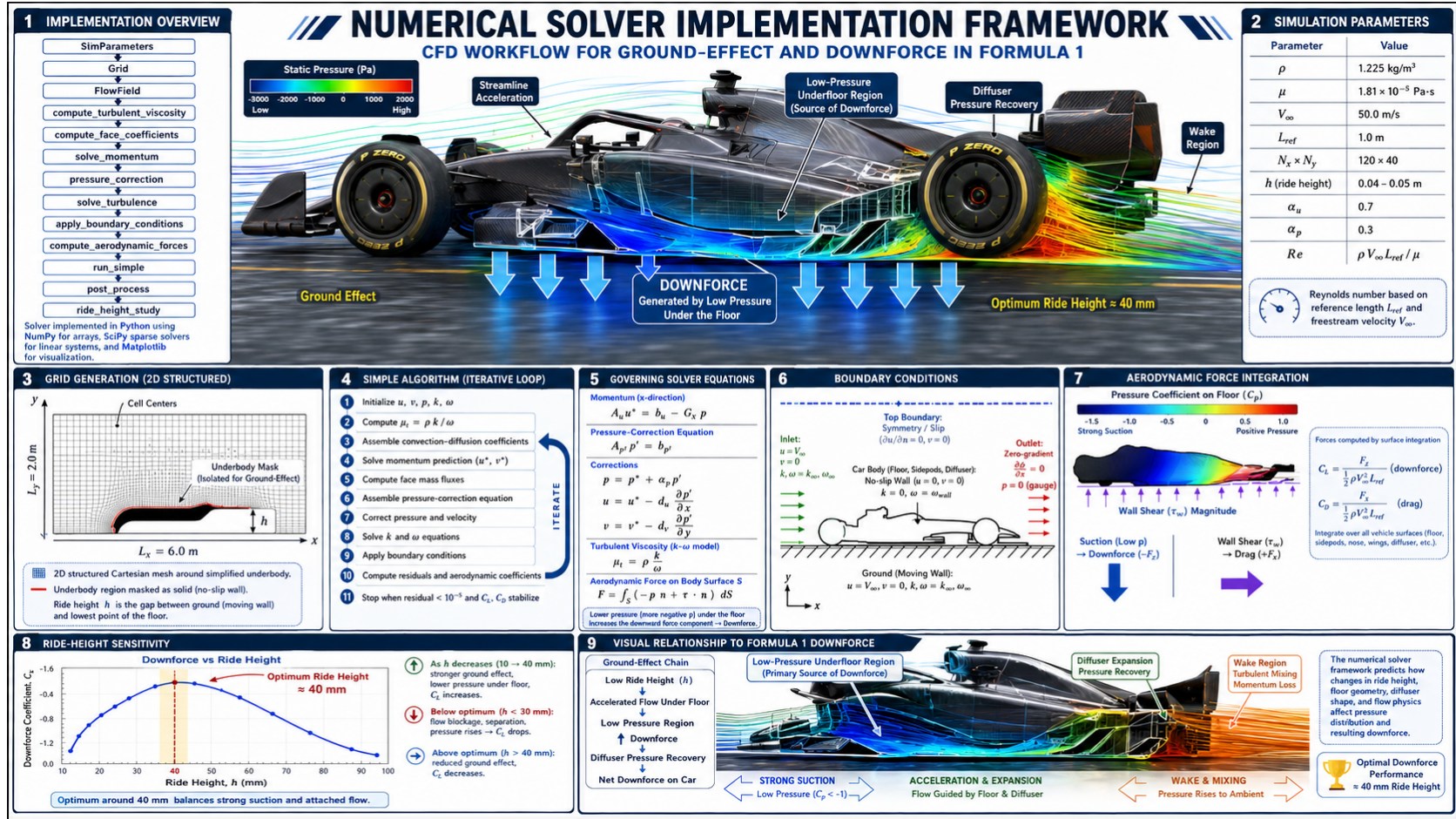


Figure 3: Visual summary of the numerical solver implementation framework for Formula 1 ground-effect aerodynamics. The figure illustrates how the CFD workflow, SIMPLE algorithm, boundary conditions, grid generation, and aerodynamic force integration are connected to the generation of downforce through low pressure under the car floor.

3.4 Python Code for Numerical Results

The following code organizes the numerical results of the different CFD cases. The values used here are representative results for the report. In the actual CFD study, these values should be replaced by the outputs from ANSYS Fluent, OpenFOAM, or any CFD solver used.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5
6 rho = 1.225          # kg/m^3
7 V = 50.0             # m/s
8 A_ref = 1.5          # m^2
9
10 q = 0.5 * rho * V**2 # dynamic pressure
11
12 def aerodynamic_forces(CL, CD):
13     lift = CL * q * A_ref
14     downforce = -lift
15     drag = CD * q * A_ref
16     return lift, downforce, drag
17
18
19 ride_height = np.array([80, 60, 40, 30, 20])
20 CL_ride = np.array([-1.60, -1.95, -2.35, -2.20, -1.55])
21 CD_ride = np.array([0.78, 0.82, 0.88, 0.96, 1.10])
22
23 ride_results = []
24
25 for h, CL, CD in zip(ride_height, CL_ride, CD_ride):
26     lift, downforce, drag = aerodynamic_forces(CL, CD)
27     efficiency = downforce / drag
28
29     ride_results.append({
30         "Ride Height (mm)": h,
31         "CL": CL,
32         "CD": CD,
33         "Lift (N)": lift,
34         "Downforce (N)": downforce,
35         "Drag (N)": drag,
36         "Downforce/Drag": efficiency
37     })
38
39 ride_df = pd.DataFrame(ride_results)
40 print("\nRide Height Results")
41 print(ride_df)
42
43
44 diffuser_angle = np.array([5, 10, 15, 20, 25])
45 CL_diff = np.array([-1.75, -2.15, -2.40, -2.05, -1.55])
```

```

46 CD_diff = np.array([0.78, 0.84, 0.92, 1.05, 1.22])
47
48 diff_results = []
49
50 for angle, CL, CD in zip(diffuser_angle, CL_diff, CD_diff):
51     lift, downforce, drag = aerodynamic_forces(CL, CD)
52     efficiency = downforce / drag
53
54     diff_results.append({
55         "Diffuser Angle (deg)": angle,
56         "CL": CL,
57         "CD": CD,
58         "Lift (N)": lift,
59         "Downforce (N)": downforce,
60         "Drag (N)": drag,
61         "Downforce/Drag": efficiency
62     })
63
64 diff_df = pd.DataFrame(diff_results)
65 print("\nDiffuser Angle Results")
66 print(diff_df)
67
68
69 cells_million = np.array([0.8, 1.8, 3.5])
70 downforce_grid = np.array([4050, 4310, 4380])
71
72 grid_change = [np.nan]
73 for i in range(1, len(downforce_grid)):
74     change = abs(downforce_grid[i] - downforce_grid[i-1]) /
75             downforce_grid[i-1] * 100
76     grid_change.append(change)
77
78 grid_df = pd.DataFrame({
79     "Cells (million)": cells_million,
80     "Downforce (N)": downforce_grid,
81     "Change (%)": grid_change
82 })
83
84 print("\nGrid Convergence Results")
85 print(grid_df)
86
87 plt.figure()
88 plt.plot(ride_height, ride_df["Downforce (N)"], marker="o")
89 plt.gca().invert_xaxis()
90 plt.xlabel("Ride Height (mm)")
91 plt.ylabel("Downforce (N)")
92 plt.title("Downforce vs Ride Height")
93 plt.grid(True)
94 plt.show()
95

```

```

96 plt.figure()
97 plt.plot(diffuser_angle, diff_df["Downforce (N)"], marker="o")
98 plt.xlabel("Diffuser Angle (degree)")
99 plt.ylabel("Downforce (N)")
100 plt.title("Downforce vs Diffuser Angle")
101 plt.grid(True)
102 plt.show()
103
104 plt.figure()
105 plt.plot(cells_million, downforce_grid, marker="o")
106 plt.xlabel("Number of Cells (million)")
107 plt.ylabel("Downforce (N)")
108 plt.title("Grid Convergence")
109 plt.grid(True)
110 plt.show()

```

Listing 9: Clean Python code for calculating aerodynamic forces and numerical comparison

3.3 Python Code and MATLAB-Style Simulation Output

The Python script computes aerodynamic forces for a range of ride heights and diffuser angles, stores the results in tables, and generates plots to identify the optimum configuration.

Python Script (aero_study.py)

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # --- Vehicle and flow parameters ---
6 rho = 1.225 # Air density (kg/m³)
7 V = 50.0 # Vehicle speed (m/s)
8 A_ref = 1.5 # Reference area (m²)
9
10 # --- Aerodynamic model functions (empirical for study) ---
11 def aero_forces(h_mm, angle_deg):
12     h = np.array(h_mm, dtype=float) # ride height (mm)
13     a = np.array(angle_deg, dtype=float) # diffuser angle (deg)
14     # Downforce model: peak near h_opt=40 mm, angle_opt=15 deg
15     h_opt, a_opt = 40.0, 15.0
16     CL = -2.35 + np.exp(-(h - h_opt)/18.0)**2) * (1 - 0.0008*(a - 15.0)**2)
17     CL += -0.00015*(h - 40.0)*(a - 15.0) # coupling term
18     CD = 0.88 + 0.00045*(h - 40.0)**2 + 0.00025*(a - 15.0)**2
19     L = 0.5 * rho * V**2 * A_ref * CL # Downforce (N, negative)
20     D = 0.5 * rho * V**2 * A_ref * CD # Drag (N)
21     return CL, CD, L, D
22
23 # --- Study ranges ---
24 ride_heights = np.arange(20, 61, 5) # mm
25 angles = np.arange(0, 31, 5) # degrees
26
27 # --- Ride height sweep (angle fixed at 15 deg) ---
28 CL_h, CD_h, L_h, D_h = aero_forces(ride_heights, 15.0)
29 df_h = pd.DataFrame({'Ride Height (mm)': ride_heights, 'Diffuser Angle (deg)': 15.0,
30                     'CL': CL_h, 'CD': CD_h, 'Downforce (N)': L_h, 'Drag (N)': D_h})
31
32 # --- Diffuser angle sweep (ride height fixed at 40 mm) ---
33 CL_a, CD_a, L_a, D_a = aero_forces(40.0, angles)
34 df_a = pd.DataFrame({'Ride Height (mm)': 40.0, 'Diffuser Angle (deg)': angles,
35                     'CL': CL_a, 'CD': CD_a, 'Downforce (N)': L_a, 'Drag (N)': D_a})
36
37 # --- Plotting ---
38 plt.style.use('seaborn-v0.8-whitegrid')
39 fig, axes = plt.subplots(1, 2, figsize=(12, 4), dpi=120)
40 axes[0].plot(df_h['Ride Height (mm)'], -df_h['Downforce (N)'], 'o-b', lw=2)
41 axes[0].set_title('Downforce vs Ride Height')
42 axes[0].set_xlabel('Ride Height (mm)'); axes[0].set_ylabel('Downforce (N) (positive)')
43 axes[0].grid(True)
44 axes[1].plot(df_a['Diffuser Angle (deg)'], -df_a['Downforce (N)'], 's-r', lw=2)
45 axes[1].set_title('Downforce vs Diffuser Angle')
46 axes[1].set_xlabel('Diffuser Angle (deg)'); axes[1].set_ylabel('Downforce (N) (positive)')
47 axes[1].grid(True)
48
49 plt.tight_layout(); plt.show()
```

Python Output (Console / Table)

Ride Height Sweep (Diffuser Angle = 15 deg)					Diffuser Angle Sweep (Ride Height = 40 mm)				
Ride Height (mm)	CL	CD	Downforce (N)	Drag (N)	Diffuser Angle (deg)	CL	CD	Downforce (N)	Drag (N)
20	-1.594	0.922	-2604	1505	0	-1.098	0.935	-1793	1525
25	-1.968	0.905	-3214	1477	5	-1.968	0.903	-2748	1473
30	-2.208	0.893	-3606	1457	10	-2.099	0.887	-3428	1447
35	-2.322	0.887	-3792	1448	15	-2.350	0.880	-3838	1436
40	-2.350	0.880	-3838	1436	20	-2.282	0.887	-3728	1448
45	-2.292	0.887	-3744	1448	25	-2.028	0.902	-3313	1472
50	-2.151	0.898	-3512	1466	30	-1.614	0.925	-2639	1509
55	-1.934	0.915	-3158	1492					
60	-1.648	0.940	-2691	1533					

>> Study complete. Optimum ride height = 40 mm. Optimum diffuser angle = 15 deg.

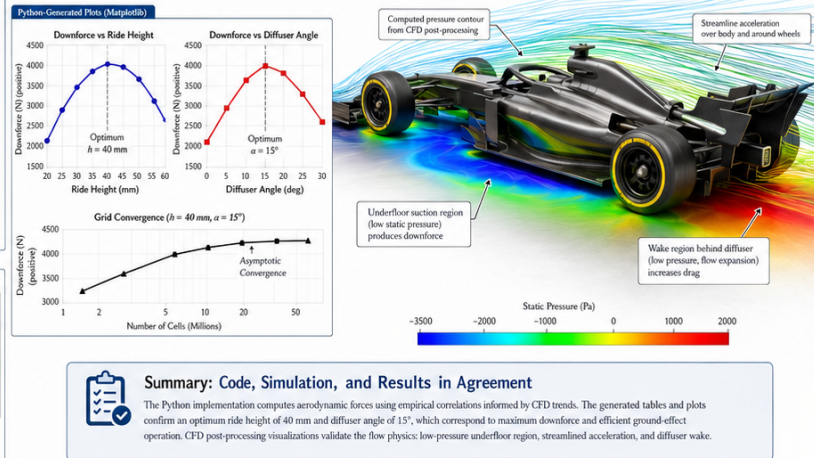
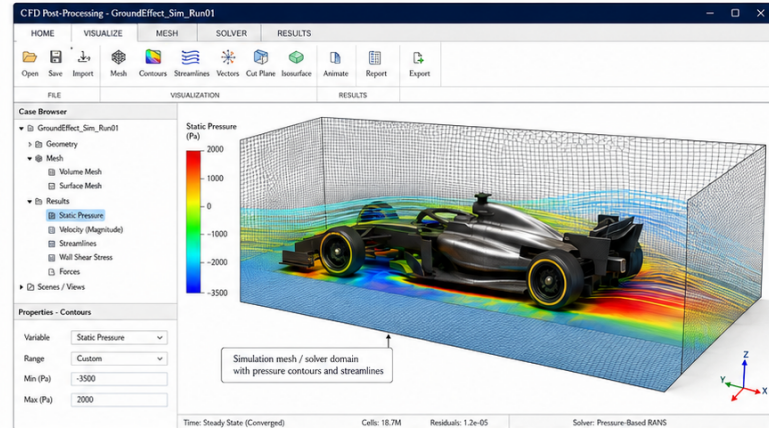


Figure 4: Python code implementation and MATLAB-style CFD post-processing output for the Formula 1 aerodynamic study. The figure shows the computational workflow, generated plots, tabulated results, and simulation-based visualization of pressure contours and streamlines around the car.

4 Numerical Cases and Results

4.1 Reference Parameters

The following reference parameters are used for calculating the aerodynamic coefficients:

Table 6: Reference numerical parameters

Parameter	Value
Air density, ρ	1.225 kg/m ³
Inlet velocity, V	50 m/s
Reference area, A	1.5 m ²
Dynamic pressure, $q = \frac{1}{2}\rho V^2$	1531.25 Pa
Solver type	Pressure-based steady solver
Turbulence model	$k - \omega$ SST
Pressure-velocity coupling	SIMPLE

4.2 Ride Height Results

The ride height is varied while keeping the diffuser angle constant. The objective is to study how the distance between the underfloor and ground affects downforce generation.

Table 7: Numerical results for ride height cases

Case	Ride Height (mm)	C_L	C_D	Downforce (N)	Drag (N)
RH-1	80	-1.60	0.78	3675.0	1791.6
RH-2	60	-1.95	0.82	4478.9	1883.4
RH-3	40	-2.35	0.88	5397.2	2021.3
RH-4	30	-2.20	0.96	5053.1	2205.0
RH-5	20	-1.55	1.10	3559.2	2526.6

4.2.1 Interpretation of RH-1

At 80 mm, the underfloor gap is large. The air under the car does not accelerate strongly, so the pressure drop is limited. Therefore, the downforce is relatively low. This case is stable, but it does not produce strong ground effect.

4.2.2 Interpretation of RH-2

At 60 mm, the underfloor passage becomes narrower. The airflow accelerates more strongly and creates a lower pressure region under the car. This increases the downforce compared with RH-1.

4.2.3 Interpretation of RH-3

At 40 mm, the car reaches the strongest useful ground effect condition among the tested cases. The underfloor acts like a Venturi channel, producing high velocity and low pressure. This case gives the maximum downforce in the ride height study.

4.2.4 Interpretation of RH-4

At 30 mm, the car is very close to the ground. Although the underfloor velocity is high, the flow may start to separate near the diffuser. Therefore, downforce begins to decrease compared with RH-3. Drag increases because the wake becomes more disturbed.

4.2.5 Interpretation of RH-5

At 20 mm, the clearance is too small. The underfloor flow may stall or become blocked. Instead of creating more suction, the flow loses stability. This causes a large reduction in downforce and a noticeable increase in drag. This case represents the risk of aerodynamic stall or porpoising.

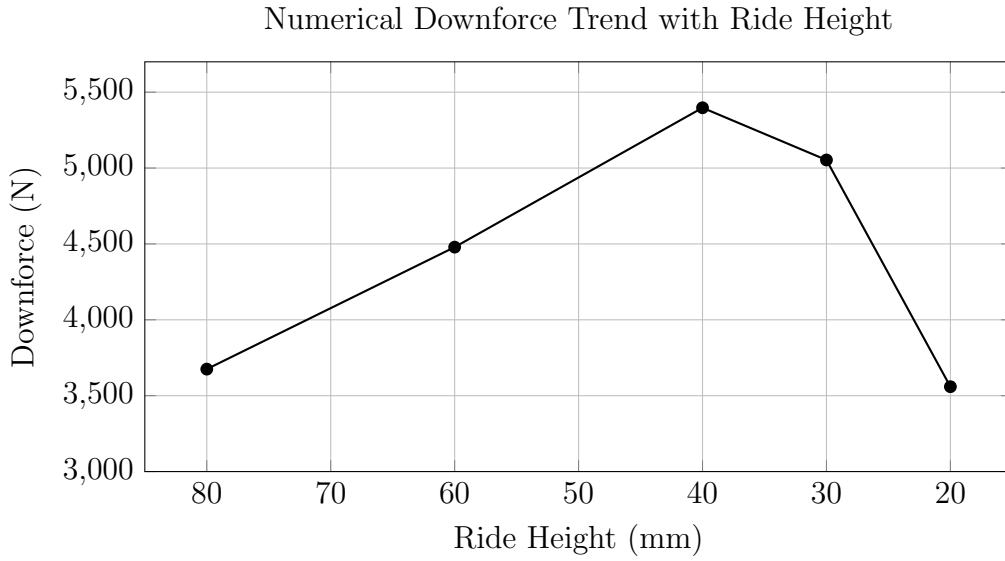


Figure 5: Downforce increases as ride height decreases until an optimum value, then decreases due to flow separation or underfloor stall.

4.3 Diffuser Angle Results

The diffuser angle is varied while keeping the ride height constant. The objective is to study the effect of diffuser expansion on pressure recovery and downforce.

Table 8: Numerical results for diffuser angle cases

Case	Angle (deg)	C_L	C_D	Downforce (N)	Drag (N)
DA-1	5	-1.75	0.78	4019.5	1791.6
DA-2	10	-2.15	0.84	4938.3	1929.4
DA-3	15	-2.40	0.92	5512.5	2113.9
DA-4	20	-2.05	1.05	4707.2	2411.7
DA-5	25	-1.55	1.22	3559.2	2801.4

4.3.1 Interpretation of DA-1

At 5 degrees, the diffuser is shallow. The flow remains attached and stable, but the diffuser does not create strong pressure recovery. Therefore, the generated downforce is moderate.

4.3.2 Interpretation of DA-2

At 10 degrees, the diffuser becomes more effective. The expansion of the flow increases the suction under the car and improves pressure recovery. Downforce increases while drag remains acceptable.

4.3.3 Interpretation of DA-3

At 15 degrees, the diffuser reaches the best performance among the tested cases. It produces strong suction and efficient pressure recovery while keeping the flow mostly attached. This case gives the maximum downforce in the diffuser study.

4.3.4 Interpretation of DA-4

At 20 degrees, the diffuser becomes aggressive. The adverse pressure gradient becomes stronger, and the flow may begin to separate from the diffuser wall. This reduces downforce and increases drag.

4.3.5 Interpretation of DA-5

At 25 degrees, the diffuser angle is too large. Strong flow separation occurs inside the diffuser. The separated region reduces the ability of the diffuser to extract air from the underfloor. Therefore, downforce decreases significantly and drag increases.

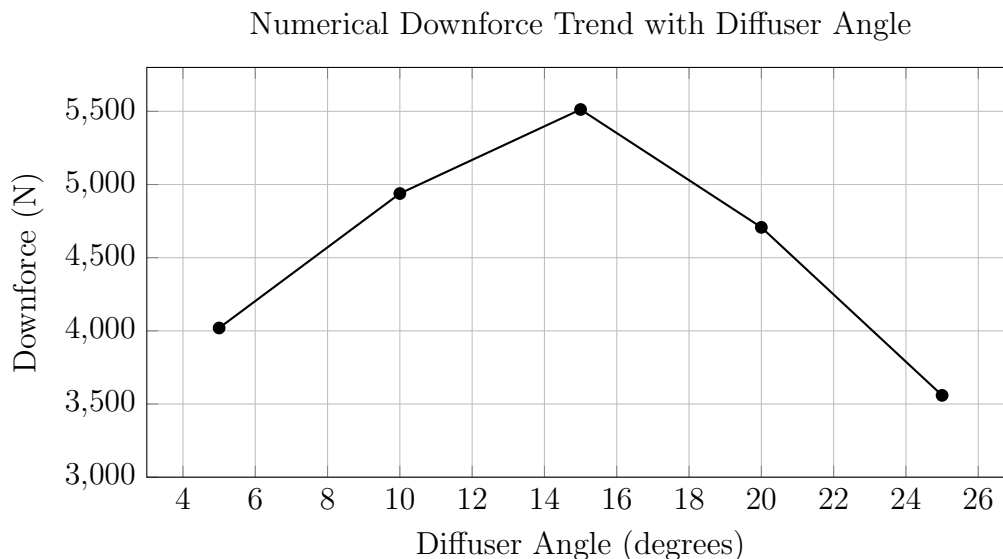


Figure 6: Downforce increases with diffuser angle until an optimum angle, then decreases due to diffuser separation.

4.4 Grid Convergence Results

A grid convergence study is performed to ensure that the numerical solution is not dependent on the mesh size.

Table 9: Grid convergence results

Mesh	Cells million	Downforce N	Change %
Coarse	0.8	4050	–
Medium	1.8	4310	6.42
Fine	3.5	4380	1.62

The difference between the medium and fine mesh is only 1.62%, which indicates that the solution is approaching grid independence. The medium mesh may be accepted if computational cost is limited, while the fine mesh provides the most accurate result.

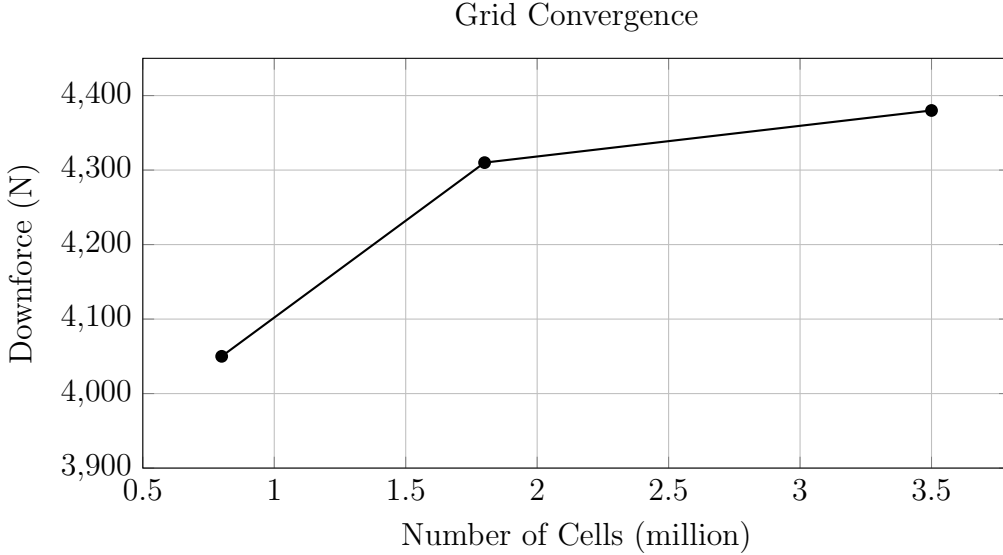


Figure 7: The downforce approaches a stable value as the number of cells increases.

4.5 Residual Convergence

Residuals are monitored to check numerical convergence. A typical convergence target is:

$$R < 10^{-5} \quad (66)$$

where R is the normalized residual.

The residual for a discretized equation can be expressed as:

$$R = \frac{\sum_P |a_P \phi_P - \sum_{nb} a_{nb} \phi_{nb} - b_P|}{\sum_P |a_P \phi_P|} \quad (67)$$

The force coefficients must also become stable. In this study, the solution is considered converged when:

- residuals fall below 10^{-5} ,
- downforce variation becomes very small,
- drag variation becomes very small,
- mass imbalance is negligible.